

Experiment ‘p_3_10_temperature_altitude’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_10_temperature_altitude

Natural language statement: Write a method that takes a temperature value and the altitude in meters. Return whether water is liquid, solid, or gaseous at the given temperature. The boiling point of water drops by about one degree centigrade for every 300 meters of altitude.

Method signature: p_3_10_temperature_altitude (t: real, a: nat) returns (s: string)

Ensures

- `if t < 0.0 then s == "solid" else if t == 0.0 then s == "solid or liquid" else if 0.0 < t < 100.0 - (a as real / 300.0) then s == "liquid" else if t == 100.0 - (a as real / 300.0) then s == "liquid or gaseous" else s == "gaseous"`

Requires

- `t >= -273.15 && a <= 10000`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a temperature value and the altitude in meters. Return whether

- ↪ water is liquid, solid, or gaseous at the given temperature. The boiling point of
- ↪ water drops by about one degree centigrade for every 300 meters of altitude.

The signature should be:

`p_3_10_temperature_altitude (t: real, a: nat) returns (s: string)`

The method should respect the following contract:

requires `t >= -273.15 && a <= 10000`, ensures if `t < 0.0` then `s == "solid"` else if `t ==`
↪ `0.0` then `s == "solid or liquid"` else if `0.0 < t < 100.0 - (a as real / 300.0)` then
↪ `s == "liquid"` else if `t == 100.0 - (a as real / 300.0)` then `s == "liquid or`
↪ `gaseous"` else `s == "gaseous"`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdrBqLEi65ZZ0cPYp1IBrsiD62r0

Model: gpt-5.5-2026-04-23

Created at: 1780485325

Finish reason: stop

Usage: 548 tokens in, and 578 tokens out

```
{
  var bp: real := 100.0 - ((a as real) / 300.0);
  if t < 0.0 {
    s := "solid";
  } else if t == 0.0 {
    s := "solid or liquid";
  } else if t < bp {
    s := "liquid";
  } else if t == bp {
    s := "liquid or gaseous";
  } else {
    s := "gaseous";
  }
}
```

Final Program

```
method p_3_10_temperature_altitude (t: real, a: nat) returns (s: string)
  requires t >= -273.15 && a <= 10000
  ensures if t < 0.0 then s == "solid" else if t == 0.0 then s == "solid or liquid"
    ↪ else if 0.0 < t < 100.0 - (a as real / 300.0) then s == "liquid" else if t
    ↪ == 100.0 - (a as real / 300.0) then s == "liquid or gaseous" else s == "
    ↪ gaseous"
{
  var bp: real := 100.0 - ((a as real) / 300.0);
  if t < 0.0 {
    s := "solid";
  } else if t == 0.0 {
    s := "solid or liquid";
  } else if t < bp {
    s := "liquid";
  } else if t == bp {
    s := "liquid or gaseous";
  } else {
    s := "gaseous";
  }
}
```

Total Token Usage

Input tokens: 548

Output tokens: 578

Reasoning tokens: 438

Sum of 'total tokens': 1126

Experiment Timings

Overall Experiment started at 1780485324860, ended at 1780485341855, lasting 16995ms (17.00 seconds)

Iteration #1 started at 1780485324861, ended at 1780485341855, lasting 16994ms (16.99 seconds)

Experiment 'p_3_1_classify_int' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_1_classify_int

Natural language statement: Write a method that takes an integer and returns whether it is negative, zero, or positive.

Method signature: p_3_1_classify_int (x: int) returns (s: string)

Ensures

- $x == 0 \implies s == \text{"zero"}$
- $x > 0 \implies s == \text{"positive"}$
- $x < 0 \implies s == \text{"negative"}$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes an integer and returns whether it is negative, zero, or
→ positive.

The signature should be:

`p_3_1_classify_int (x: int) returns (s: string)`

The method should respect the following contract:

`ensures x == 0 ==> s == "zero", ensures x > 0 ==> s == "positive", ensures x < 0 ==> s ==`
→ `"negative"`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdjR98ZBhumS1axeV4dvclHUP05o

Model: gpt-5.5-2026-04-23

Created at: 1780484845

Finish reason: stop

Usage: 320 tokens in, and 136 tokens out

```
{  
  if x == 0 {  
    s := "zero";  
  } else if x > 0 {  
    s := "positive";  
  } else {  
    s := "negative";  
  }  
}
```

Final Program

```
method p_3_1_classify_int (x: int) returns (s: string)  
  ensures x == 0 ==> s == "zero"  
  ensures x > 0 ==> s == "positive"  
  ensures x < 0 ==> s == "negative"  
{
```

```
if x == 0 {  
  s := "zero";  
} else if x > 0 {  
  s := "positive";  
} else {  
  s := "negative";  
}  
}
```

Total Token Usage

Input tokens: 320

Output tokens: 136

Reasoning tokens: 66

Sum of 'total tokens': 456

Experiment Timings

Overall Experiment started at 1780484844575, ended at 1780484852890, lasting 8315ms (8.32 seconds)

Iteration #1 started at 1780484844580, ended at 1780484852890, lasting 8310ms (8.31 seconds)

Experiment ‘p_3_2_classify_real’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_2_classify_real

Natural language statement: Write a method that takes a floating point number and returns whether it is negative, zero, or positive. Add "small" if the absolute value of the number is less than 1, or "large" if it exceeds 1,000,000.

Method signature: p_3_2_classify_real (x: real) returns (s: string)

Ensures

- $x == 0.0 \implies s == \text{"zero, small"}$
- $1.0 > x > 0.0 \implies s == \text{"positive, small"}$
- $1.0 \leq x < 1000000.0 \implies s == \text{"positive"}$
- $1000000.0 \leq x \implies s == \text{"positive, large"}$
- $0.0 > x > -1.0 \implies s == \text{"negative, small"}$
- $-1.0 \geq x > -1000000.0 \implies s == \text{"negative"}$
- $-1000000.0 \geq x \implies s == \text{"negative, large"}$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a floating point number and returns whether it is negative,
→ zero, or positive. Add "small" if the absolute value of the number is less than 1,
→ or "large" if it exceeds 1,000,000.

The signature should be:

`p_3_2_classify_real (x: real) returns (s: string)`

The method should respect the following contract:

`ensures x == 0.0 ==> s == "zero, small", ensures 1.0 > x > 0.0 ==> s == "positive, small`
→ `", ensures 1.0 <= x < 1000000.0 ==> s == "positive", ensures 1000000.0 <= x ==> s`
→ `== "positive, large", ensures 0.0 > x > -1.0 ==> s == "negative, small", ensures`
→ `-1.0 >= x > -1000000.0 ==> s == "negative", ensures -1000000.0 >= x ==> s == "`
→ `negative, large"`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdjZD52VuE9RxsuUxeyFvix9x5ca

Model: gpt-5.5-2026-04-23

Created at: 1780484853

Finish reason: stop

Usage: 580 tokens in, and 593 tokens out

```
{
  if x == 0.0 {
    s := "zero, small";
  } else if x > 0.0 {
    if x < 1.0 {
      s := "positive, small";
    } else if x < 1000000.0 {
      s := "positive";
    } else {
      s := "positive, large";
    }
  } else {
    if x > -1.0 {
      s := "negative, small";
    } else if x > -1000000.0 {
```

```
    s := "negative";
  } else {
    s := "negative, large";
  }
}
}
```

Final Program

```
method p_3_2_classify_real (x: real) returns (s: string)
  ensures x == 0.0 ==> s == "zero, small"
  ensures 1.0 > x > 0.0 ==> s == "positive, small"
  ensures 1.0 <= x < 1000000.0 ==> s == "positive"
  ensures 1000000.0 <= x ==> s == "positive, large"
  ensures 0.0 > x > -1.0 ==> s == "negative, small"
  ensures -1.0 >= x > -1000000.0 ==> s == "negative"
  ensures -1000000.0 >= x ==> s == "negative, large"
{
  if x == 0.0 {
    s := "zero, small";
  } else if x > 0.0 {
    if x < 1.0 {
      s := "positive, small";
    } else if x < 1000000.0 {
      s := "positive";
    } else {
      s := "positive, large";
    }
  } else {
    if x > -1.0 {
      s := "negative, small";
    } else if x > -1000000.0 {
      s := "negative";
    } else {
      s := "negative, large";
    }
  }
}
```

Total Token Usage

Input tokens: 580

Output tokens: 593

Reasoning tokens: 413

Sum of 'total tokens': 1173

Experiment Timings

Overall Experiment started at 1780484853556, ended at 1780484868258, lasting 14702ms (14.70 seconds)

Iteration #1 started at 1780484853557, ended at 1780484868257, lasting 14700ms (14.70 seconds)

Experiment ‘p_3_3_number_of_digits’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_3_3_number_of_digits

Natural language statement: Write a method that takes an integer and returns how many digits the number has, by checking whether the number is 10, 100, and so on. (Assume that all integers are less than ten billion.) If the number is negative, first multiply it with -1 .

Method signature: p_3_3_number_of_digits (i: int) returns (n: nat)

Ensures

- $i \geq 0 \implies n == \text{digits}(i)$
- $i < 0 \implies n == \text{digits}(-i)$

Requires

- $10000000000 > i > -10000000000$

Functional Code Given

```
function digits (n: nat): nat
{
  if 0 <= n <= 9 then 1
  else 1 + digits (n / 10)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes an integer and returns how many digits the number has, by
↪ checking whether the number is 10, 100, and so on. (Assume that all integers are
↪ less than ten billion.) If the number is negative, first multiply it with -1.

The signature should be:

`p_3_3_number_of_digits (i: int) returns (n: nat)`

The method should respect the following contract:

`requires 10000000000 > i > -100000000000, ensures i >= 0 ==> n == digits(i), ensures i < 0`
↪ `==> n == digits(-i)`

The contract uses the following dafny code:

```
function digits (n: nat): nat
{
  if 0 <= n <= 9 then 1
  else 1 + digits (n / 10)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdjpai0BG6Llant3OpgIiediJzhJ

Model: gpt-5.5-2026-04-23

Created at: 1780484869

Finish reason: stop

Usage: 528 tokens in, and 2967 tokens out

```
{
  var x: int := i;
  if x < 0 {
    x := -x;
  }

  assert 0 <= x;
  assert x < 10000000000;
```

```

if x >= 1000000000 {
  assert 1000000000 <= x < 10000000000;
  n := 10;
} else if x >= 100000000 {
  assert 100000000 <= x < 1000000000;
  n := 9;
} else if x >= 10000000 {
  assert 10000000 <= x < 100000000;
  n := 8;
} else if x >= 1000000 {
  assert 1000000 <= x < 10000000;
  n := 7;
} else if x >= 100000 {
  assert 100000 <= x < 1000000;
  n := 6;
} else if x >= 10000 {
  assert 10000 <= x < 100000;
  n := 5;
} else if x >= 1000 {
  assert 1000 <= x < 10000;
  n := 4;
} else if x >= 100 {
  assert 100 <= x < 1000;
  n := 3;
} else if x >= 10 {
  assert 10 <= x < 100;
  n := 2;
} else {
  assert 0 <= x < 10;
  n := 1;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_3_3_number_of_digits/program.dfy(11,0): Error: a postcondition could not
  ↪ be proved on this return path

```

```

|
11 | {
| ^

```

```

problem_data/p_3_3_number_of_digits/program.dfy(9,22): Related location: this is the
  ↪ postcondition that could not be proved

```

```

|
9 | ensures i >= 0 ==> n == digits(i)
| ^^

```



```
problem_data/p_3_3_number_of_digits/program.dfy(11,0): Error: a postcondition could not
  ↳ be proved on this return path
```

```
  |
11 | {
  | ^
```

```
problem_data/p_3_3_number_of_digits/program.dfy(10,21): Related location: this is the
  ↳ postcondition that could not be proved
```

```
  |
10 | ensures i < 0 ==> n == digits(-i)
  | ^^
```

Dafny program verifier finished with 2 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the

- ↳ Dafny body, including the curly braces that surround it. Do not show the signature
- ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdl3Iwvzq9F7vxfyELT34n26dSgg

Model: gpt-5.5-2026-04-23

Created at: 1780484945

Finish reason: stop

Usage: 1218 tokens in, and 6025 tokens out

```
{
  var x: int;
  if i < 0 {
    x := -i;
  } else {
    x := i;
  }

  assert 0 <= x;
  assert x < 10000000000;
  var y: nat := x;

  var a0: nat := y;
  var a1: nat := a0 / 10;
  var a2: nat := a1 / 10;
  var a3: nat := a2 / 10;
  var a4: nat := a3 / 10;
  var a5: nat := a4 / 10;
  var a6: nat := a5 / 10;
  var a7: nat := a6 / 10;
  var a8: nat := a7 / 10;
  var a9: nat := a8 / 10;
```

```

if y >= 1000000000 {
  n := 10;
  assert 100000000 <= a1 < 1000000000;
  assert 10000000 <= a2 < 100000000;
  assert 1000000 <= a3 < 10000000;
  assert 100000 <= a4 < 1000000;
  assert 10000 <= a5 < 100000;
  assert 1000 <= a6 < 10000;
  assert 100 <= a7 < 1000;
  assert 10 <= a8 < 100;
  assert 0 <= a9 <= 9;
  assert digits(a9) == 1;
  assert digits(a8) == 1 + digits(a9);
  assert digits(a8) == 2;
  assert digits(a7) == 1 + digits(a8);
  assert digits(a7) == 3;
  assert digits(a6) == 1 + digits(a7);
  assert digits(a6) == 4;
  assert digits(a5) == 1 + digits(a6);
  assert digits(a5) == 5;
  assert digits(a4) == 1 + digits(a5);
  assert digits(a4) == 6;
  assert digits(a3) == 1 + digits(a4);
  assert digits(a3) == 7;
  assert digits(a2) == 1 + digits(a3);
  assert digits(a2) == 8;
  assert digits(a1) == 1 + digits(a2);
  assert digits(a1) == 9;
  assert digits(a0) == 1 + digits(a1);
  assert digits(a0) == 10;
  assert n == digits(a0);
} else if y >= 100000000 {
  n := 9;
  assert 10000000 <= a1 < 100000000;
  assert 1000000 <= a2 < 10000000;
  assert 100000 <= a3 < 1000000;
  assert 10000 <= a4 < 100000;
  assert 1000 <= a5 < 10000;
  assert 100 <= a6 < 1000;
  assert 10 <= a7 < 100;
  assert 0 <= a8 <= 9;
  assert digits(a8) == 1;
  assert digits(a7) == 1 + digits(a8);
  assert digits(a7) == 2;
  assert digits(a6) == 1 + digits(a7);
  assert digits(a6) == 3;
  assert digits(a5) == 1 + digits(a6);
  assert digits(a5) == 4;
  assert digits(a4) == 1 + digits(a5);
  assert digits(a4) == 5;
  assert digits(a3) == 1 + digits(a4);

```

```

    assert digits(a3) == 6;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 7;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 8;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 9;
    assert n == digits(a0);
} else if y >= 10000000 {
    n := 8;
    assert 1000000 <= a1 < 10000000;
    assert 100000 <= a2 < 1000000;
    assert 10000 <= a3 < 100000;
    assert 1000 <= a4 < 10000;
    assert 100 <= a5 < 1000;
    assert 10 <= a6 < 100;
    assert 0 <= a7 <= 9;
    assert digits(a7) == 1;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 2;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 3;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 4;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 5;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 6;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 7;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 8;
    assert n == digits(a0);
} else if y >= 1000000 {
    n := 7;
    assert 100000 <= a1 < 1000000;
    assert 10000 <= a2 < 100000;
    assert 1000 <= a3 < 10000;
    assert 100 <= a4 < 1000;
    assert 10 <= a5 < 100;
    assert 0 <= a6 <= 9;
    assert digits(a6) == 1;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 2;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 3;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 4;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 5;
    assert digits(a1) == 1 + digits(a2);

```

```

    assert digits(a1) == 6;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 7;
    assert n == digits(a0);
} else if y >= 100000 {
    n := 6;
    assert 10000 <= a1 < 100000;
    assert 1000 <= a2 < 10000;
    assert 100 <= a3 < 1000;
    assert 10 <= a4 < 100;
    assert 0 <= a5 <= 9;
    assert digits(a5) == 1;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 2;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 3;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 4;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 5;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 6;
    assert n == digits(a0);
} else if y >= 10000 {
    n := 5;
    assert 1000 <= a1 < 10000;
    assert 100 <= a2 < 1000;
    assert 10 <= a3 < 100;
    assert 0 <= a4 <= 9;
    assert digits(a4) == 1;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 2;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 3;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 4;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 5;
    assert n == digits(a0);
} else if y >= 1000 {
    n := 4;
    assert 100 <= a1 < 1000;
    assert 10 <= a2 < 100;
    assert 0 <= a3 <= 9;
    assert digits(a3) == 1;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 2;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 3;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 4;

```

```

    assert n == digits(a0);
} else if y >= 100 {
    n := 3;
    assert 10 <= a1 < 100;
    assert 0 <= a2 <= 9;
    assert digits(a2) == 1;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 2;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 3;
    assert n == digits(a0);
} else if y >= 10 {
    n := 2;
    assert 0 <= a1 <= 9;
    assert digits(a1) == 1;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 2;
    assert n == digits(a0);
} else {
    n := 1;
    assert 0 <= a0 <= 9;
    assert digits(a0) == 1;
    assert n == digits(a0);
}

assert a0 == y;
assert n == digits(y);
if i < 0 {
    assert x == -i;
    assert y == -i;
    assert n == digits(-i);
} else {
    assert x == i;
    assert y == i;
    assert n == digits(i);
}
}

```

Final Program

```

function digits (n: nat): nat
{
    if 0 <= n <= 9 then 1
    else 1 + digits (n / 10)
}

method p_3_3_number_of_digits (i: int) returns (n: nat)
    requires 100000000000 > i > -100000000000
    ensures i >= 0 ==> n == digits(i)
    ensures i < 0 ==> n == digits(-i)

```

```

{
  var x: int;
  if i < 0 {
    x := -i;
  } else {
    x := i;
  }

  assert 0 <= x;
  assert x < 10000000000;
  var y: nat := x;

  var a0: nat := y;
  var a1: nat := a0 / 10;
  var a2: nat := a1 / 10;
  var a3: nat := a2 / 10;
  var a4: nat := a3 / 10;
  var a5: nat := a4 / 10;
  var a6: nat := a5 / 10;
  var a7: nat := a6 / 10;
  var a8: nat := a7 / 10;
  var a9: nat := a8 / 10;

  if y >= 10000000000 {
    n := 10;
    assert 1000000000 <= a1 < 10000000000;
    assert 10000000 <= a2 < 1000000000;
    assert 1000000 <= a3 < 100000000;
    assert 100000 <= a4 < 10000000;
    assert 10000 <= a5 < 1000000;
    assert 1000 <= a6 < 100000;
    assert 100 <= a7 < 10000;
    assert 10 <= a8 < 1000;
    assert 0 <= a9 <= 9;
    assert digits(a9) == 1;
    assert digits(a8) == 1 + digits(a9);
    assert digits(a8) == 2;
    assert digits(a7) == 1 + digits(a8);
    assert digits(a7) == 3;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 4;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 5;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 6;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 7;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 8;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 9;
  }
}

```

```

    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 10;
    assert n == digits(a0);
} else if y >= 100000000 {
    n := 9;
    assert 10000000 <= a1 < 100000000;
    assert 1000000 <= a2 < 10000000;
    assert 100000 <= a3 < 1000000;
    assert 10000 <= a4 < 100000;
    assert 1000 <= a5 < 10000;
    assert 100 <= a6 < 1000;
    assert 10 <= a7 < 100;
    assert 0 <= a8 <= 9;
    assert digits(a8) == 1;
    assert digits(a7) == 1 + digits(a8);
    assert digits(a7) == 2;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 3;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 4;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 5;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 6;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 7;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 8;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 9;
    assert n == digits(a0);
} else if y >= 10000000 {
    n := 8;
    assert 1000000 <= a1 < 10000000;
    assert 100000 <= a2 < 1000000;
    assert 10000 <= a3 < 100000;
    assert 1000 <= a4 < 10000;
    assert 100 <= a5 < 1000;
    assert 10 <= a6 < 100;
    assert 0 <= a7 <= 9;
    assert digits(a7) == 1;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 2;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 3;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 4;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 5;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 6;

```

```

    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 7;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 8;
    assert n == digits(a0);
} else if y >= 1000000 {
    n := 7;
    assert 100000 <= a1 < 1000000;
    assert 10000 <= a2 < 100000;
    assert 1000 <= a3 < 10000;
    assert 100 <= a4 < 1000;
    assert 10 <= a5 < 100;
    assert 0 <= a6 <= 9;
    assert digits(a6) == 1;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 2;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 3;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 4;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 5;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 6;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 7;
    assert n == digits(a0);
} else if y >= 100000 {
    n := 6;
    assert 10000 <= a1 < 100000;
    assert 1000 <= a2 < 10000;
    assert 100 <= a3 < 1000;
    assert 10 <= a4 < 100;
    assert 0 <= a5 <= 9;
    assert digits(a5) == 1;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 2;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 3;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 4;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 5;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 6;
    assert n == digits(a0);
} else if y >= 10000 {
    n := 5;
    assert 1000 <= a1 < 10000;
    assert 100 <= a2 < 1000;
    assert 10 <= a3 < 100;

```



```

    assert 0 <= a4 <= 9;
    assert digits(a4) == 1;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 2;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 3;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 4;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 5;
    assert n == digits(a0);
} else if y >= 1000 {
    n := 4;
    assert 100 <= a1 < 1000;
    assert 10 <= a2 < 100;
    assert 0 <= a3 <= 9;
    assert digits(a3) == 1;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 2;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 3;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 4;
    assert n == digits(a0);
} else if y >= 100 {
    n := 3;
    assert 10 <= a1 < 100;
    assert 0 <= a2 <= 9;
    assert digits(a2) == 1;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 2;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 3;
    assert n == digits(a0);
} else if y >= 10 {
    n := 2;
    assert 0 <= a1 <= 9;
    assert digits(a1) == 1;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 2;
    assert n == digits(a0);
} else {
    n := 1;
    assert 0 <= a0 <= 9;
    assert digits(a0) == 1;
    assert n == digits(a0);
}

assert a0 == y;
assert n == digits(y);
if i < 0 {

```

```
    assert x == -i;
    assert y == -i;
    assert n == digits(-i);
} else {
    assert x == i;
    assert y == i;
    assert n == digits(i);
}
}
```

Total Token Usage

Input tokens: 1746

Output tokens: 8992

Reasoning tokens: 6080

Sum of ‘total tokens’: 10738

Experiment Timings

Overall Experiment started at 1780484869015, ended at 1780485039151, lasting 170136ms (170.14 seconds)

Iteration #1 started at 1780484869017, ended at 1780484945156, lasting 76139ms (76.14 seconds)

Iteration #2 started at 1780484945157, ended at 1780485039151, lasting 93994ms (93.99 seconds)

Experiment ‘p_3_4_classification’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_4_classification

Natural language statement: Write a method that takes three numbers and returns "all the same" if they are all the same, "all different" if they are all different, and "neither" otherwise.

Method signature: p_3_4_classification (i: int, j: int, k: int) returns (s: string)

Ensures

- if `i == j == k` then `s == "all the same"` else if `i != j && j != k && i != k` then `s == "all different"` else `s == "neither"`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes three numbers and returns "all the same" if they are all the same, "all different" if they are all different, and "neither" otherwise.

The signature should be:

`p_3_4_classification (i: int, j: int, k: int) returns (s: string)`

The method should respect the following contract:

`ensures if i == j == k then s == "all the same" else if i != j && j != k && i != k then s == "all different" else s == "neither"`

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdpyOzQ7Y4QEtwlnQ2Wjn7VQPHaf

Model: gpt-5.5-2026-04-23

Created at: 1780485250

Finish reason: stop

Usage: 390 tokens in, and 214 tokens out

```
{
  if i == j && j == k {
    s := "all the same";
  } else if i != j && j != k && i != k {
    s := "all different";
  } else {
    s := "neither";
  }
}
```

Final Program

```
method p_3_4_classification (i: int, j: int, k: int) returns (s: string)
  ensures if i == j == k then s == "all the same" else if i != j && j != k && i != k
    then s == "all different" else s == "neither"
{
  if i == j && j == k {
```

```
    s := "all the same";  
  } else if i != j && j != k && i != k {  
    s := "all different";  
  } else {  
    s := "neither";  
  }  
}
```

Total Token Usage

Input tokens: 390

Output tokens: 214

Reasoning tokens: 130

Sum of 'total tokens': 604

Experiment Timings

Overall Experiment started at 1780485249397, ended at 1780485258997, lasting 9600ms (9.60 seconds)

Iteration #1 started at 1780485249404, ended at 1780485258996, lasting 9592ms (9.59 seconds)

Experiment 'p_3_5_order' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_5_order

Natural language statement: Write a method that takes three numbers and returns "increasing" if they are in increasing order, "decreasing" if they are in decreasing order, and "neither" otherwise. Here, "increasing" means "strictly increasing", with each value larger than its predecessor. The sequence 3 4 4 would not be considered increasing.

Method signature: p_3_5_order (i: int, j: int, k: int) returns (s: string)

Ensures

- if $i < j < k$ then $s == \text{"increasing"}$ else if $i > j \ \&\& \ j > k$ then $s == \text{"decreasing"}$ else $s == \text{"neither"}$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes three numbers and returns "increasing" if they are in
→ increasing order, "decreasing" if they are in decreasing order, and "neither"
→ otherwise. Here, "increasing" means "strictly increasing", with each value larger
→ than its predecessor. The sequence 3 4 4 would not be considered increasing.

The signature should be:

`p_3_5_order (i: int, j: int, k: int) returns (s: string)`

The method should respect the following contract:

`ensures if i < j < k then s == "increasing" else if i > j && j > k then s == "decreasing"`
→ `else s == "neither"`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdq8fnO0IldT2K6w3TrRQ8mmtDEg

Model: gpt-5.5-2026-04-23

Created at: 1780485260

Finish reason: stop

Usage: 446 tokens in, and 227 tokens out

```
{
  if i < j && j < k {
    s := "increasing";
  } else if i > j && j > k {
    s := "decreasing";
  } else {
    s := "neither";
  }
}
```

Final Program

```
method p_3_5_order (i: int, j: int, k: int) returns (s: string)
  ensures if i < j < k then s == "increasing" else if i > j && j > k then s == "
    → decreasing" else s == "neither"
```

```
{  
  if i < j && j < k {  
    s := "increasing";  
  } else if i > j && j > k {  
    s := "decreasing";  
  } else {  
    s := "neither";  
  }  
}
```

Total Token Usage

Input tokens: 446

Output tokens: 227

Reasoning tokens: 148

Sum of 'total tokens': 673

Experiment Timings

Overall Experiment started at 1780485260061, ended at 1780485270481, lasting 10420ms (10.42 seconds)

Iteration #1 started at 1780485260061, ended at 1780485270481, lasting 10420ms (10.42 seconds)

Experiment ‘p_3_6_parametrised_order’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_6_parametrised_order

Natural language statement: Write a method that takes three numbers and returns "increasing" if they are in increasing order, "decreasing" if they are in decreasing order, and "neither" otherwise. A Boolean parameter triggers the method to be "strict" or "lenient". In lenient mode, the sequence 3 4 4 is increasing and the sequence 4 4 4 is both increasing and decreasing.

Method signature: p_3_6_parametrised_order (i: int, j: int, k: int, strict: bool) returns (s: string)

Ensures

- if strict then if i < j < k then s == "increasing" else if i > j && j > k then s == "decreasing" else s == "neither" else if i <= j <= k then if i == j == k then s == "increasing and decreasing" else s == "increasing" else if i >= j && j >= k then s == "decreasing" else s == "neither"

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes three numbers and returns "increasing" if they are in
→ increasing order, "decreasing" if they are in decreasing order, and "neither"
→ otherwise. A Boolean parameter triggers the method to be "strict" or "lenient". In
→ lenient mode, the sequence 3 4 4 is increasing and the sequence 4 4 4 is both
→ increasing and decreasing.

The signature should be:

`p_3_6_parametrised_order (i: int, j: int, k: int, strict: bool ) returns (s: string)`

The method should respect the following contract:

ensures if strict then if $i < j < k$ then $s == \text{"increasing"}$ else if $i > j \ \&\& \ j > k$ then s
→ $== \text{"decreasing"}$ else $s == \text{"neither"}$ else if $i \leq j \leq k$ then if $i == j == k$ then s
→ $== \text{"increasing and decreasing"}$ else $s == \text{"increasing"}$ else if $i \geq j \ \&\& \ j \geq k$
→ then $s == \text{"decreasing"}$ else $s == \text{"neither"}$

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdqJ8ckgwmaHY66FcRedH7Yd9sJE

Model: gpt-5.5-2026-04-23

Created at: 1780485271

Finish reason: stop

Usage: 596 tokens in, and 693 tokens out

```
{
  if strict {
    if i < j && j < k {
      s := "increasing";
    } else if i > j && j > k {
      s := "decreasing";
    } else {
      s := "neither";
    }
  } else {
    if i <= j && j <= k {
      if i == j && j == k {
        s := "increasing and decreasing";
      } else {
```

```

        s := "increasing";
    }
} else if i >= j && j >= k {
    s := "decreasing";
} else {
    s := "neither";
}
}
}

```

Final Program

```

method p_3_6_parametrised_order (i: int, j: int, k: int, strict: bool ) returns (s:
    ↪ string)
    ensures if strict then if i < j < k then s == "increasing" else if i > j && j > k
        ↪ then s == "decreasing" else s == "neither" else if i <= j <= k then if i ==
        ↪ j == k then s == "increasing and decreasing" else s == "increasing" else
        ↪ if i >= j && j >= k then s == "decreasing" else s == "neither"
{
    if strict {
        if i < j && j < k {
            s := "increasing";
        } else if i > j && j > k {
            s := "decreasing";
        } else {
            s := "neither";
        }
    } else {
        if i <= j && j <= k {
            if i == j && j == k {
                s := "increasing and decreasing";
            } else {
                s := "increasing";
            }
        } else if i >= j && j >= k {
            s := "decreasing";
        } else {
            s := "neither";
        }
    }
}
}

```

Total Token Usage

Input tokens: 596

Output tokens: 693

Reasoning tokens: 512

Sum of 'total tokens': 1289

Experiment Timings

Overall Experiment started at 1780485271283, ended at 1780485287247, lasting 15964ms (15.96 seconds)

Iteration #1 started at 1780485271284, ended at 1780485287247, lasting 15963ms (15.96 seconds)

Experiment 'p_3_7_order' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_7_order

Natural language statement: Write a method that takes three integers and returns "in order" if they are sorted in ascending or descending order, or "not in order" otherwise.

Method signature: p_3_7_order (i: int, j: int, k: int) returns (s: string)

Ensures

- if (i < j < k) || (i > j > k) then s == "in order" else s == "not in order"

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes three integers and returns "in order" if they are sorted in
→ ascending or descending order, or "not in order" otherwise.

The signature should be:

`p_3_7_order (i: int, j: int, k: int) returns (s: string)`

The method should respect the following contract:

`ensures if (i < j < k) || (i > j > k) then s == "in order" else s == "not in order"`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdqaSmeTtOReue7IC8pghFKKbOd3

Model: gpt-5.5-2026-04-23

Created at: 1780485288

Finish reason: stop

Usage: 354 tokens in, and 208 tokens out

```
{
  if (i < j && j < k) || (i > j && j > k) {
    s := "in order";
  } else {
    s := "not in order";
  }
}
```

Final Program

```
method p_3_7_order (i: int, j: int, k: int) returns (s: string)
  ensures if (i < j < k) || (i > j > k) then s == "in order" else s == "not in order"
  → "
{
  if (i < j && j < k) || (i > j && j > k) {
    s := "in order";
  } else {
    s := "not in order";
  }
}
```

```
}  
}
```

Total Token Usage

Input tokens: 354

Output tokens: 208

Reasoning tokens: 138

Sum of ‘total tokens’: 562

Experiment Timings

Overall Experiment started at 1780485288103, ended at 1780485296583, lasting 8480ms (8.48 seconds)

Iteration #1 started at 1780485288104, ended at 1780485296583, lasting 8479ms (8.48 seconds)

Experiment 'p_3_8_two_pairs' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_8_two_pairs

Natural language statement: Write a method that takes four integers and returns "two pairs" if the input consists of two matching pairs (in some order) and "not two pairs" otherwise.

Method signature: p_3_8_two_pairs (i: int, j: int, k: int, l: int) returns (s: string)

Ensures

- if (i == j && k == l) || (i == k && j == l) || (i == l && j == k) then s == "two pairs"
else s == "not two pairs"

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes four integers and returns "two pairs" if the input consists of
→ two matching pairs (in some order) and "not two pairs" otherwise.

The signature should be:

`p_3_8_two_pairs (i: int, j: int, k: int, l: int) returns (s: string)`

The method should respect the following contract:

ensures if (i == j && k == l) || (i == k && j == l) || (i == l && j == k) then s == "two
→ pairs" else s == "not two pairs"

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdqjs5XMU5UGNziXag7sLewXPICi

Model: gpt-5.5-2026-04-23

Created at: 1780485297

Finish reason: stop

Usage: 398 tokens in, and 191 tokens out

```
{
  if (i == j && k == l) || (i == k && j == l) || (i == l && j == k) {
    s := "two pairs";
  } else {
    s := "not two pairs";
  }
}
```

Final Program

```
method p_3_8_two_pairs (i: int, j: int, k: int, l: int) returns (s: string)
  ensures if (i == j && k == l) || (i == k && j == l) || (i == l && j == k) then s
    → == "two pairs" else s == "not two pairs"
{
  if (i == j && k == l) || (i == k && j == l) || (i == l && j == k) {
    s := "two pairs";
  } else {
```

```
s := "not two pairs";  
}  
}
```

Total Token Usage

Input tokens: 398

Output tokens: 191

Reasoning tokens: 111

Sum of 'total tokens': 589

Experiment Timings

Overall Experiment started at 1780485297267, ended at 1780485308594, lasting 11327ms (11.33 seconds)

Iteration #1 started at 1780485297268, ended at 1780485308594, lasting 11326ms (11.33 seconds)

Experiment 'p_3_9_temperature' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_3_9_temperature

Natural language statement: Write a method that takes a temperature value and a letter C for Celsius or F for Fahrenheit. Return whether water is liquid, solid, or gaseous at the given temperature at sea level.

Method signature: p_3_9_temperature (t: real, c: char) returns (s: string)

Ensures

- `if c == 'C' then if t < 0.0 then s == "solid" else if t == 0.0 then s == "solid or liquid" else if 0.0 < t < 100.0 then s == "liquid" else if t == 100.0 then s == "liquid or gaseous" else s == "gaseous" else if t < 32.0 then s == "solid" else if t == 32.0 then s == "solid or liquid" else if 32.0 < t < 212.0 then s == "liquid" else if t == 212.0 then s == "liquid or gaseous" else s == "gaseous"`

Requires

- `(c == 'C' && t >= -273.15) || (c == 'F' && t >= -459.67)`
- `c == 'C' || c == 'F'`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a temperature value and a letter C for Celsius or F for
→ Fahrenheit. Return whether water is liquid, solid, or gaseous at the given
→ temperature at sea level.

The signature should be:

p_3_9_temperature (t: real, c: char) returns (s: string)

The method should respect the following contract:

```
requires (c == 'C' && t >= -273.15) || (c == 'F' && t >= -459.67), requires c == 'C' || c
→ == 'F', ensures if c == 'C' then if t < 0.0 then s == "solid" else if t == 0.0
→ then s == "solid or liquid" else if 0.0 < t < 100.0 then s == "liquid" else if t
→ == 100.0 then s == "liquid or gaseous" else s == "gaseous" else if t < 32.0 then s
→ == "solid" else if t == 32.0 then s == "solid or liquid" else if 32.0 < t < 212.0
→ then s == "liquid" else if t == 212.0 then s == "liquid or gaseous" else s == "
→ gaseous"
```

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdqym04i4sVl3YV7odeZcL1pAYlw

Model: gpt-5.5-2026-04-23

Created at: 1780485309

Finish reason: stop

Usage: 696 tokens in, and 597 tokens out

```
{
  if c == 'C' {
    if t < 0.0 {
      s := "solid";
    } else if t == 0.0 {
      s := "solid or liquid";
    } else if t < 100.0 {
      s := "liquid";
    } else if t == 100.0 {
      s := "liquid or gaseous";
    } else {
      s := "gaseous";
    }
  }
}
```

```

} else {
  if t < 32.0 {
    s := "solid";
  } else if t == 32.0 {
    s := "solid or liquid";
  } else if t < 212.0 {
    s := "liquid";
  } else if t == 212.0 {
    s := "liquid or gaseous";
  } else {
    s := "gaseous";
  }
}
}
}

```

Final Program

```

method p_3_9_temperature (t: real, c: char) returns (s: string)
  requires (c == 'C' && t >= -273.15) || (c == 'F' && t >= -459.67)
  requires c == 'C' || c == 'F'
  ensures if c == 'C' then if t < 0.0 then s == "solid" else if t == 0.0 then s == "
    ↪ solid or liquid" else if 0.0 < t < 100.0 then s == "liquid" else if t ==
    ↪ 100.0 then s == "liquid or gaseous" else s == "gaseous" else if t < 32.0
    ↪ then s == "solid" else if t == 32.0 then s == "solid or liquid" else if
    ↪ 32.0 < t < 212.0 then s == "liquid" else if t == 212.0 then s == "liquid or
    ↪ gaseous" else s == "gaseous"
{
  if c == 'C' {
    if t < 0.0 {
      s := "solid";
    } else if t == 0.0 {
      s := "solid or liquid";
    } else if t < 100.0 {
      s := "liquid";
    } else if t == 100.0 {
      s := "liquid or gaseous";
    } else {
      s := "gaseous";
    }
  } else {
    if t < 32.0 {
      s := "solid";
    } else if t == 32.0 {
      s := "solid or liquid";
    } else if t < 212.0 {
      s := "liquid";
    } else if t == 212.0 {
      s := "liquid or gaseous";
    } else {
      s := "gaseous";
    }
  }
}

```

```
}  
}  
}
```

Total Token Usage

Input tokens: 696

Output tokens: 597

Reasoning tokens: 354

Sum of ‘total tokens’: 1293

Experiment Timings

Overall Experiment started at 1780485309334, ended at 1780485324115, lasting 14781ms (14.78 seconds)

Iteration #1 started at 1780485309337, ended at 1780485324115, lasting 14778ms (14.78 seconds)

